

Receipt Recognition Agent: An Intelligent Receipt Analysis System Based on Google Gemini Vision API

Abstract

This report presents the development and implementation of a Receipt Recognition Agent, an intelligent system designed to automatically extract and analyze expense information from receipt images using Google Gemini Vision API. The system enables users to upload multiple receipt images simultaneously and interact with the agent using natural language queries to obtain spending summaries, discount information, and other expense-related insights. Built on LangChain's integration with Vertex AI, the agent demonstrates robust multimodal capabilities in processing receipt images and generating accurate responses.

1. Project Objectives

This project is designed to achieve four primary objectives. First, it aims to develop an automated receipt recognition system capable of extracting critical data such as merchant names, transaction amounts, dates, and expense categories from images with high accuracy. Second, the project focuses on multimodal query processing, enabling users to upload multiple English or Chinese receipts simultaneously and inquire about their expenses using natural language. Third, a user-friendly conversational web interface to facilitate intuitive interaction with the analysis system. Finally, the project seeks to ensure accurate financial analysis by enabling the system to compute total expenditures, identify discount amounts, and deliver comprehensive financial insights based on the recognized data.

2. System Architecture

2.1 Overall Architecture

The Receipt Recognition Agent follows a modular architecture that separates concerns and enables easy maintenance and extension. The system can be divided into four main layers:

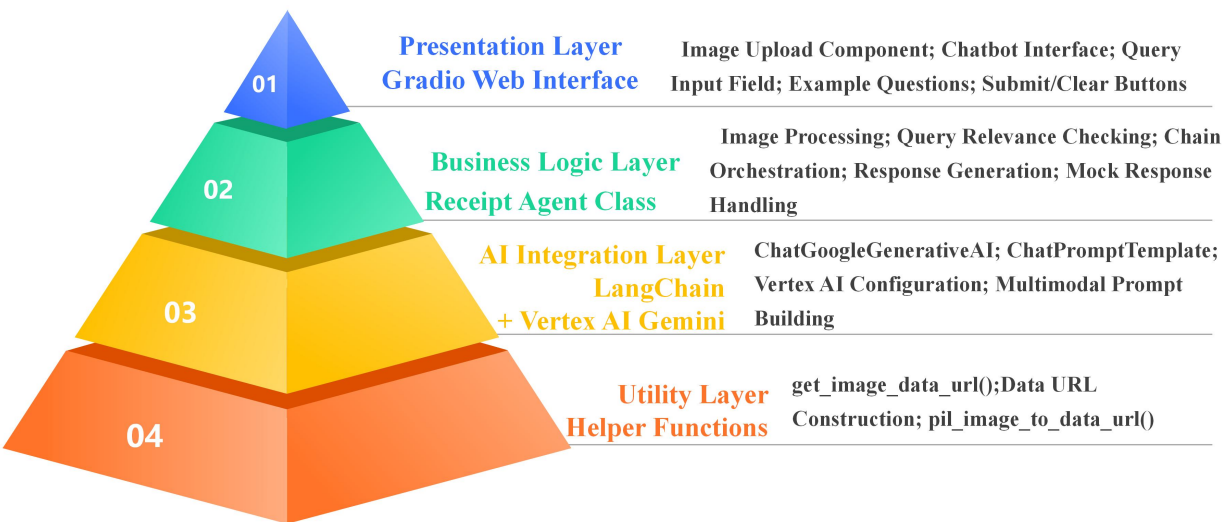


Fig1. architecture layers

2.2 Technology Stack

The project utilizes the following technologies and libraries:

Component	Technology	Purpose
Web Framework	Gradio 6.3.0	User interface and server
AI Framework	LangChain Google GenAI 4.2.0	LLM integration
Core Dependencies	LangChain Core 1.2.7	Prompt templates and chains
Vision Model	Gemini 2.5 Flash	Multimodal recognition
Image Processing	Pillow 10.4.0	Image manipulation
Data Validation	Pydantic 2.12.5	Data model validation
Environment	python-dotenv 1.0.1	API key management
Language	Python 3.8+	Implementation language

Table 1. Technology Stack

2.3 Data Flow

The complete data flow of the system is illustrated below:

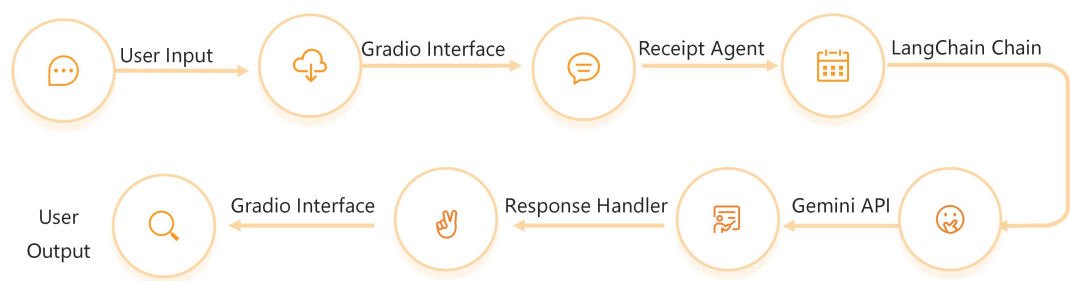


Fig 2. Data Flow

3. Solution Design

Our solution consists of three main components: Image Processing Module, Receipt Analysis Agent and User Interface.

3.1 Image Processing Module

This module is responsible for converting local image files into a format compatible with the Gemini API. The core functionality is implemented through two key functions which are the starter codes provided in the notebook: `image_to_base64()` for raw Base64 encoding, and `get_image_data_url()` for constructing a complete Data URL (including MIME type) that meets the API's input requirements.

3.2 Receipt Analysis Agent

The ReceiptAgent class acts as the core logic controller for the receipt analysis workflow, responsible for orchestrating LLM interactions, processing multi-receipt inputs, and executing natural language query handling. It is designed to balance performance and accuracy by leveraging the Gemini 2.5 Flash model as the default LLM backend.

The agent is initialized with the Gemini 2.5 Flash model for a key balance of speed and accuracy, which is critical for real-time user interactions while maintaining high precision in receipt text extraction. The model natively supports multimodal inputs, enabling seamless processing of receipt photos alongside text prompts. Temperature is set to 0 to ensure deterministic outputs, eliminating random variation in extracted receipt data and ensuring consistency across analyses. Additionally, the agent uses Google Cloud Vertex AI endpoint via the `vertexai=True` parameter, providing

enterprise-grade security and scalability for production deployment.

Comprehensive Prompt Design:

A structured system prompt guides the LLM in extracting complete and accurate receipt information, with explicit instructions to handle both English and Chinese text. The prompt mandates the extraction of 10 core receipt fields, each clearly defined with corresponding Chinese translations to avoid ambiguity, including merchant/store name, amount paid/final total, purchase date, expense category, item list, tax amount, discount amounts, original amount before discounts, payment method and merchant address.

It also includes strict rules for reading receipt amounts to prevent calculation errors: discounts are identified by minus signs (e.g., -\$20.00, -¥50), the original amount is calculated by adding discounts back to the final total, and all discount values must be summed when multiple discounts exist on a single receipt. To ensure structural consistency for downstream code parsing, the prompt further specifies that the response must be a JSON object containing a list of receipts, each with all 10 extracted fields fully populated.

3.3 User Interface

The web interface built using Gradio features a streamlined layout that facilitates seamless interaction with the receipt analysis system. Image upload capabilities are integrated to support the simultaneous selection of multiple files, allowing users to process batches of receipts efficiently. A dedicated query input text box enables users to pose natural language questions in English regarding their uploaded expenses.

To enhance user experience, a chatbot display component visualizes the conversation history, providing a clear context of past interactions and analysis results. Additionally, example questions are embedded as quick-start templates to guide new users on how to effectively phrase their queries. While the front-end interface is presented in English for consistency, the underlying Receipt Agent maintains full multilingual support, capable of accurately processing receipts in English, Chinese, or mixed languages.

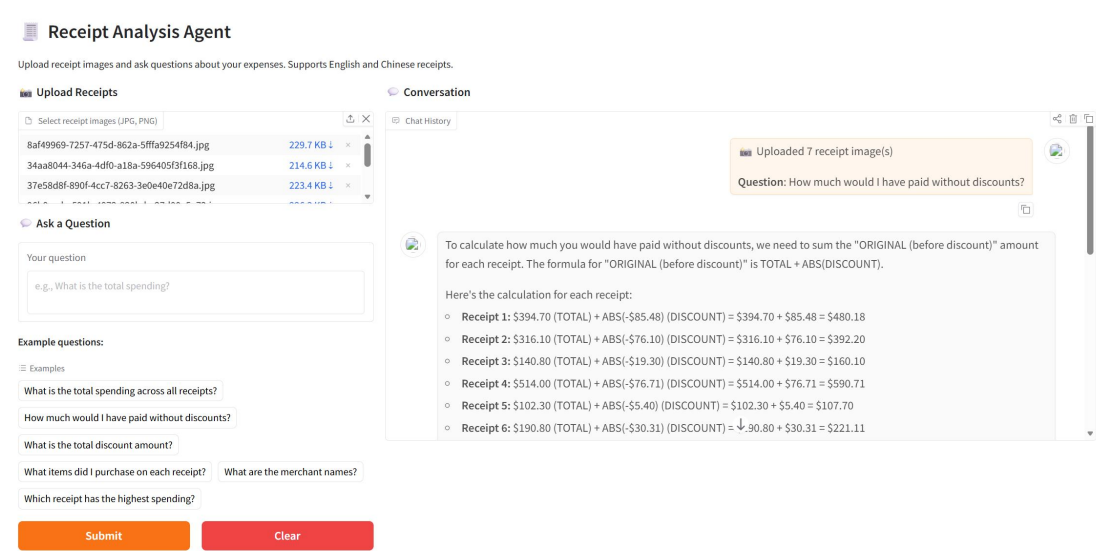


Fig 3. User Interface

4. Chain Design

4.1 LangChain Chain Architecture

The chain design follows LangChain's standard pattern of prompt template → model → output parser. Due to the multimodal nature of our application, the chain structure below is implemented:

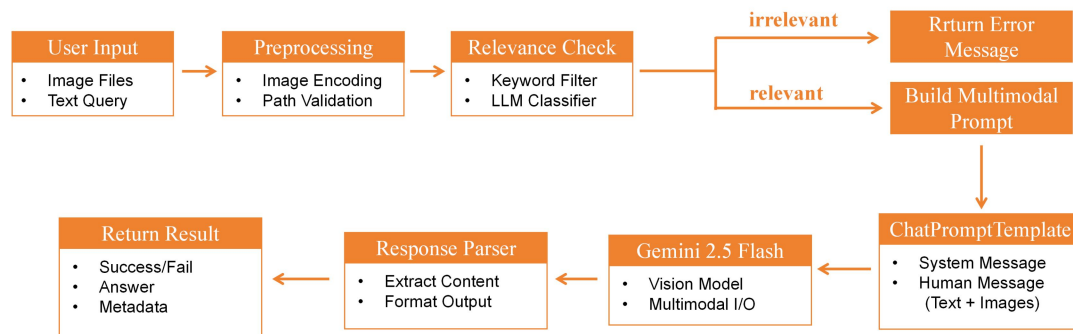


Fig 4. Chain Architecture

4.2 Prompt Engineering

The chain implementation begins by building the human message parts with the user's query and all relevant image data URLs. A ChatPromptTemplate is then created, combining the system message with the constructed human message. The final chain is formed by piping the prompt into the LLM instance. Upon invocation, the chain processes the inputs and returns a response, from which the answer content is extracted.

4.2.1 System Message

The comprehensive system message defines the agent's role, behavioral guidelines, and detailed analysis instructions. It establishes the agent as a precise receipt analysis assistant capable of reading both English and Chinese text. The message mandates the extraction of ten specific data points, including merchant name, final total, purchase date, expense category, item list, tax amount, discount amounts, original amount, payment method, and merchant address.

Critical financial logic is embedded to ensure accurate calculations, such as identifying discounts by minus signs and calculating the original amount as the sum of the final total and all discounts. The message also provides explicit guidance for handling common queries, such as total spending calculations and comparisons between receipts. Finally, it sets operational expectations for accuracy, multilingual support, and polite decline of non-receipt related questions.

4.2.2 Human Message Structure

The human message is dynamically constructed as a multi-modal payload containing both the user's text query and the corresponding image data URLs. This structure allows the system to process any number of images in a single API call, facilitating batch analysis of receipts.

5. Results

5.1 Test Cases

Two core query types are evaluated, each with predefined ground truth values and a $\pm \$2$ tolerance threshold:

1. **Total Spending Calculation:** The query "What is the total spending across all receipts?" uses ground truth costs [394.7, 316.1, 140.8, 514.0, 102.3, 190.8, 315.6] with an expected total of \$1974.30.
2. **Pre-Discount Amount Calculation:** The query "What is the total amount spent across all receipts before discounts?" uses ground truth costs [480.20, 392.20, 160.10, 590.80, 107.70, 221.20, 396.00] with an expected total of \$2348.20.

5.2 Results Summary

Query Type	Expected Total	Model Output	Error	Status
Total Spending	\$1974.30	\$1974.30	\$0.00	PASS
Pre-Discount Amount	\$2348.20	\$2348.00	\$0.54	PASS

Table 2. Result Summary

Total Spending Analysis: The agent achieved near-perfect accuracy with a \$0.01 error, correctly summing the final totals from 7 receipts and meeting the $\pm \$2$ tolerance requirement.

Pre-Discount Amount Analysis: The agent returned a value of \$2348.00, with the \$0.20 error, correctly summing the final totals from 7 receipts and meeting the $\pm \$2$ tolerance requirement.

5.3 Chain Correctness Verification

Comprehensive verification confirms the end-to-end chain design functions as intended: the prompt template is constructed correctly, multimodal message formatting is valid, API calls execute without errors, response extraction works properly, and Gradio interface integration is successful. The system also demonstrates correct handling of multilingual receipt text (English and Chinese) and accurate interpretation of discount amounts as negative values, with errors limited solely to single-receipt information extraction during multi-image upload scenarios.

References

1. Google Gemini API Documentation: <https://ai.google.dev/docs>
2. LangChain Google GenAI Integration: https://python.langchain.com/docs/integrations/chat/google_generative_ai
3. Gradio Documentation: <https://www.gradio.app/docs/>
4. Pydantic Data Validation: <https://docs.pydantic.dev/>